

132. Les collisions

Regarder **AVANT** de bouger, et non reculer après.

1. Ce qu'il faut comprendre

`lire(colonne, ligne)` rend le numéro de la tuile affichée. Comparer à `MUR` suffit : **le nom d'une tuile EST son numéro**.

La forme compte. On calcule la position **voulue**, on regarde ce qu'il y a, et l'on n'y va que si c'est libre. Bouger puis reculer marche aussi, mais laisse une image où le personnage est dans le mur — et cette image finit toujours par se voir.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **500 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	222 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AdresseCase	10 o	← poser(), lire() — trouve l'adresse d'une case de l'écran
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	40 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	32 o	← CopierTuiles — 1 lettre sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

poser()	× 5	55 o sur place, dans main() + AdresseCase (10 o)
sprite()	× 1	29 o sur place, dans main()
bouton()	× 4	20 o sur place, dans main() + LireManette (37 o)
lire()	× 1	11 o sur place, dans main() + AdresseCase (10 o)
image()	× 1	6 o sur place, dans main() + AttendreImage (28 o)

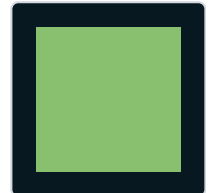
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : une salle entourée de murs, avec un mur au milieu.
// Les flèches déplacent un A case par case ; il ne traverse jamais un mur.
//
// La méthode : on calcule d'abord la case VISÉE (vc, vl), on la lit avec
// lire(colonne, ligne), et on ne bouge que si ce n'est pas un mur.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <poser>    // pose une tuile sur une case du fond
#include <bouton>   // lit un bouton de la manette
#include <lire>     // lit la tuile posée sur une case
#include <sprite>   // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — MUR

```
Tuile MUR = {
    "#####", "#-----#", "#-----#", "#-----#",
    "#-----#", "#-----#", "#-----#", "#####",
};
```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le jeu commence ici.
    for (uint8_t y = 0; y < 18; y++) { poser(0, y, MUR); poser(19, y, MUR); } // gauche et droite
    for (uint8_t x = 0; x < 20; x++) { poser(x, 0, MUR); poser(x, 17, MUR); } // haut et bas
    poser(10, 8, MUR); // un mur au milieu

    uint8_t c = 5; // La case du héros : colonne...
    uint8_t l = 8; // ... et ligne.

    while (true) { // Répète pour toujours :
        uint8_t vc = c; // La case visée : au départ, la même.
        uint8_t vl = l;

        if (bouton(DROITE)) vc++; // Les flèches changent la case visée,
        if (bouton(GAUCHE)) vc--; // pas encore le héros.
        if (bouton(BAS)) vl++;
        if (bouton(HAUT)) vl--;

        // On regarde AVANT de bouger : la case visée est-elle libre ?
        if (lire(vc, vl) != MUR) { c = vc; l = vl; } // Libre : le héros y va.
                                                // Un mur : il reste où il est.

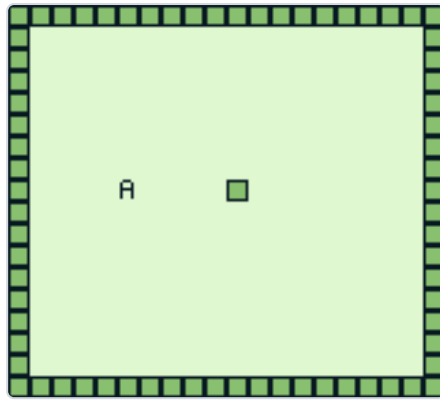
        sprite(0, c * 8, l * 8, 1); // Le lutin : case × 8 = pixel. Dessin : la tuile 1 (le A).
        image(); // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



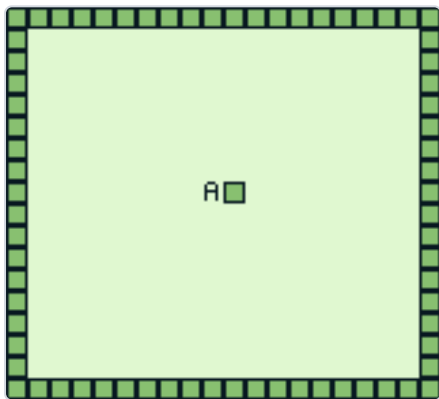
la toute première image — la console vient de démarrer — image 1



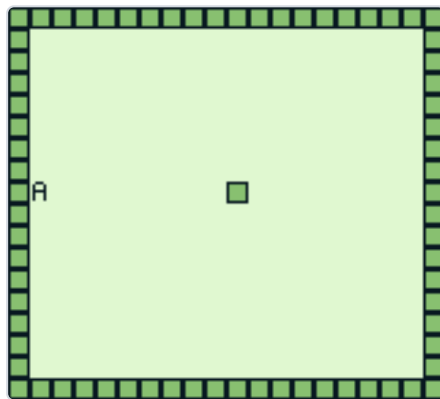
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

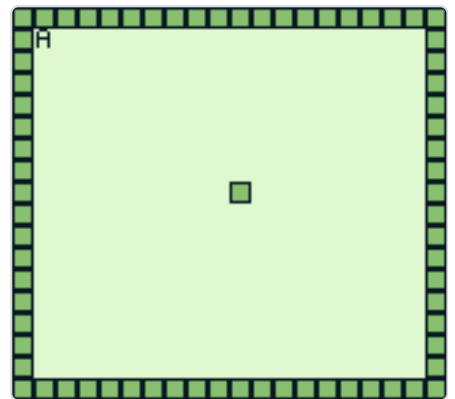
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



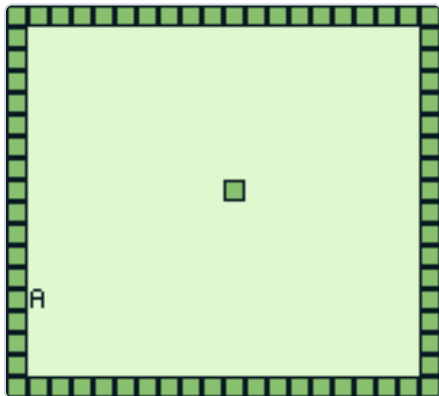
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le curseur s'arrête net contre les murs et contre le bloc isolé.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ il s'arrête devant le bloc du milieu (colonne 9)

✓ il ne le traverse pas

✓ il s'arrête sous le mur du haut (ligne 1)

✓ et devant celui de gauche (colonne 1)

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

MUR

+ tuile 8 × 8

+ perso 16 × 16

+ perso 32 × 32

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

MUR

tuile n° 44 — poser(x, y, MUR)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

×2

÷2

Agrandir...

Vider le dessin

Taille voulue :

✓ Créer la copie

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

MUR ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

MUR peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

Ensuite : **133. Ralentir sans compter les images à la main** — chapitre 23 : Un vrai jeu. · gameboy3 — écrire une cartouche Game Boy en C++